

Special Topics in Discrete Mathematics: Advanced Topics in Cryptography

Theory Exercise 3

Panayi Andreas^{*}
MSc: A.L.MA.
Student ID: 711514240008

School of Electrical and Computer Engineering,
National Technical University of Athens
Athens, Greece

April 2026



National Technical
University of Athens

^{*}Writing was done using $\text{\LaTeX}$

Contents

Problem 1	1
Description	1
Solution	1
Problem 2	2
Description	2
Solution	2
Exercise 3	4
Description	4
Solution	4
Exercise 4	6
Description	6
Solution	6
Exercise 5	8
Description	8
Solution	8

Problem 1

Description

In class, we studied *binary* Merkle trees, but Merkle trees can have higher dimensions. A d -dimensional Merkle tree is a *complete* tree in which every non-leaf node has exactly d children. The value of a leaf is the hash of its contents, as in a binary Merkle tree. The value of an internal node is the hash of the concatenation of its children's values. For example, in a ternary Merkle tree constructed using the hash function H , an internal node with value v which has children with values v_1, v_2 , and v_3 will take the value $v = H(v_1 \| v_2 \| v_3)$.

Consider a ternary tree constructed using blake2s, a hash function with an output of $\kappa = 256$ bits which has 3^7 leaves. Calculate the proof size, in bytes, for this Merkle tree. You can assume the verifier already has the Merkle tree root and has received the claimed contents and index of the leaf node in question, so you do not need to include these in your proof size calculation.

Solution

General case.

Consider a complete d -ary Merkle tree with $n = d^h$ leaves, where each hash output has length κ bits. The height of the tree is

$$h = \log_d n.$$

To prove the inclusion of a leaf, the verifier already knows the leaf contents, the leaf index, and the Merkle root. Therefore, the proof only needs to contain the sibling hashes along the path from the leaf to the root.

At each level, the current node has $d - 1$ siblings. Hence the proof contains

$$(d - 1)h = (d - 1) \log_d n$$

hash values. Since each hash has size $\kappa/8$ bytes, the proof size is

$$(d - 1) \log_d n \cdot \frac{\kappa}{8} \text{ bytes.}$$

Our case.

Here the tree is ternary, so $d = 3$, it has $n = 3^7$ leaves, and blake2s has output length

$$\kappa = 256 \text{ bits.}$$

Thus the height is

$$h = \log_3(3^7) = 7.$$

At each level of a ternary tree, the proof must include the other

$$d - 1 = 2$$

sibling hashes. Therefore, the total number of hashes in the proof is

$$2 \cdot 7 = 14.$$

Each hash has size

$$\frac{256}{8} = 32 \text{ bytes.}$$

Hence the total proof size is

$$14 \cdot 32 = 448 \text{ bytes.}$$

Problem 2

Description

In class, we studied Merkle trees where the leaves are relatively few. Many modern blockchain applications make use of sparse Merkle trees. A sparse Merkle tree is a complete binary Merkle tree with exactly 2^κ leaves. Since the tree has an exponential number of leaves, they cannot be processed by a polynomially bound computer. However, the vast majority of these leaves have contents set to the empty string, whereas only a polynomial number of leaves have non-empty contents. This makes computing the root and proofs within these trees efficiently possible.

Consider a sparse Merkle tree with 2^κ leaves constructed using sha256, a hash function with an output of $\kappa = 256$ in which the first leaf has contents equal to the string hello and value equal to the hash

$$2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824.$$

All the other $2^{256} - 1$ leaves of this tree have their contents set to the empty string and their value is equal to

$$e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855.$$

What is the value of the root of this tree?

Solution

Algorithm.

Let H denote sha256. Define

$$a_0 = H(\text{hello})$$

and

$$e_0 = H(\varepsilon),$$

where ε is the empty string.

For $i = 0, 1, \dots, 255$, compute

$$a_{i+1} = H(a_i \parallel e_i)$$

and

$$e_{i+1} = H(e_i \parallel e_i).$$

After these 256 iterations, output a_{256} .

Why this is sufficient.

The tree has 2^{256} leaves, but only the first leaf is non-empty. Therefore, at every level, the left subtree contains the leaf hello, while the right subtree is completely empty.

Thus, to compute the root, it is enough to compute two values at each level:

$$a_i = \text{the root of the subtree containing hello,}$$

and

$$e_i = \text{the root of a completely empty subtree of height } i.$$

All completely empty subtrees of the same height have the same root, so one value e_i represents all of them.

At level $i + 1$, the subtree containing hello is obtained by combining the current hello-subtree with an empty sibling subtree:

$$a_{i+1} = H(a_i \parallel e_i).$$

Similarly, a completely empty subtree of height $i + 1$ is obtained by combining two completely empty subtrees of height i :

$$e_{i+1} = H(e_i \parallel e_i).$$

Hence the two sequences

$$a_0, a_1, \dots, a_{256}$$

and

$$e_0, e_1, \dots, e_{256}$$

are sufficient to compute the root.

Complexity.

The initial values a_0 and e_0 are given in the problem. Each of the 256 iterations computes two hashes:

$$H(a_i \parallel e_i) \quad \text{and} \quad H(e_i \parallel e_i).$$

Therefore, the algorithm performs $2 \cdot 256 = 512$ hash computations.

It stores only the two current hash values a_i and e_i . Since each sha256 hash is 256 bits, i.e. 32 bytes, the working storage for these hash values is $2 \cdot 32 = 64$ bytes, ignoring loop counters and implementation overhead.

Python computation.

The following code computes the sparse Merkle root. The concatenations are performed on raw bytes, and only the final value is printed in hexadecimal form.

```
1  import hashlib
2
3  def H(x: bytes) -> bytes:
4      return hashlib.sha256(x).digest()
5
6  a = bytes.fromhex(
7      "2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824"
8  )
9
10 e = bytes.fromhex(
11     "e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855"
12 )
13
14 for _ in range(256):
15     a, e = H(a + e), H(e + e)
16
17 print(a.hex())
18
```

Running this code gives the root

f1a36a9f4e9d7824a2826b4a078ad8b5463dec600e96726c6a3487b2c7bf7112

Exercise 3

Description

Consider a bitcoin backbone execution with $n = 10$, $t = 2$, $q = 1000$, $\kappa = 256$, and $T = 2^{\kappa-19}$. Numerically calculate the following quantities:

1. The upper bound of the honest advantage δ .
2. The probability p of a successful query.
3. The probability f that a round is successful.
4. The expectation $\mathbb{E}[Z_r]$ of the number of successful adversarial queries in a round.
5. The expectation $\mathbb{E}[Z(S)]$ of the number of successful adversarial queries in a set of consecutive rounds with $|S| = 256$.
6. Your best lower bound for $\mathbb{E}[Y_r]$, the expectation of a round being a convergence opportunity.

Solution

We are given $n = 10$, $t = 2$, $q = 1000$, $\kappa = 256$, and $T = 2^{\kappa-19}$.

1. Honest advantage. The honest majority condition is $t < (1-\delta)(n-t)$. In our case, $2 < 8(1-\delta)$, so $\delta < 1 - \frac{2}{8} = \frac{3}{4}$. Hence the upper bound is

$$\delta_{\max} = \frac{3}{4} = 0.75.$$

2. Probability of a successful query. A single query succeeds with probability $p = T/2^\kappa$. Since $T = 2^{\kappa-19}$, we get

$$p = \frac{2^{\kappa-19}}{2^\kappa} = 2^{-19} = \frac{1}{524288} \approx 1.9073486328125 \cdot 10^{-6}.$$

3. Probability that a round is successful. A round is successful if at least one honest query succeeds. The honest parties make $q(n-t) = 1000 \cdot 8 = 8000$ queries per round, so

$$f = 1 - (1-p)^{8000} = 1 - (1 - 2^{-19})^{8000}.$$

Numerically,

$$f \approx 0.0151429779393149.$$

4. Expected number of successful adversarial queries in a round. The adversary makes $qt = 1000 \cdot 2 = 2000$ queries per round. Therefore,

$$\mathbb{E}[Z_r] = qtp = 2000 \cdot 2^{-19} = 0.003814697265625.$$

5. Expected number of successful adversarial queries in 256 rounds. For a set S of consecutive rounds with $|S| = 256$, linearity of expectation gives

$$\mathbb{E}[Z(S)] = |S|\mathbb{E}[Z_r] = 256 \cdot 0.003814697265625 = 0.9765625.$$

6. Lower bound for $\mathbb{E}[Y_r]$. A sufficient condition for a convergence opportunity is that exactly one honest query succeeds in the round. There are $q(n - t) = 8000$ honest queries, each succeeding with probability $p = 2^{-19}$. Thus

$$\mathbb{E}[Y_r] = \Pr[Y_r = 1] \geq 8000 \cdot 2^{-19} (1 - 2^{-19})^{7999}.$$

Numerically,

$$\mathbb{E}[Y_r] \geq 0.0150277542.$$

Exercise 4

Description

Consider *bitcoin backbone executions* with $n = 10$, $t = 2$, $q = 1000$, and a hash function with $\kappa = 256$ bits. Suppose that the probability of an execution of interest not being typical is at most

$$4 \cdot 10^{12} e^{-\varepsilon^2 \lambda f / 3} + 4 \cdot 10^{20} 2^{-\kappa}.$$

Numerically calculate:

1. The upper bound of the honest advantage δ .
2. A secure parametrization for the probability f of a successful round and the Chernoff error ε that respects the balancing inequality.
3. The probability p of a successful query.
4. A suitable target T to achieve it.
5. A Chernoff interval parameter λ that ensures typical executions occur with probability at least $1 - 2^{-128}$.
6. The chain growth parameters: the chain growth interval s and the chain velocity τ .
7. The common prefix parameter k .
8. The chain quality parameters: the chunk size ℓ and the chain quality μ .

Solution

We are given $n = 10$, $t = 2$, $q = 1000$, and $\kappa = 256$.

1. Honest advantage. The honest majority condition is $t < (1-\delta)(n-t)$. In our case, $2 < 8(1-\delta)$, and hence $\delta < 1 - \frac{2}{8} = \frac{3}{4}$. Therefore, the upper bound is $\delta_{\max} = 3/4 = 0.75$. In the numerical calculations below we use the limiting value $\delta = 3/4$.

2. Choice of f and ε . The balancing inequality is $3\varepsilon + 3f \leq \delta$. We choose $\varepsilon = f = 0.1$. Then

$$3\varepsilon + 3f = 3 \cdot 0.1 + 3 \cdot 0.1 = 0.6 \leq 0.75 = \delta,$$

so this choice is valid.

3. Probability of a successful query. A round is successful if at least one honest query succeeds. Since the honest parties make $q(n-t) = 1000 \cdot 8 = 8000$ queries per round, we have

$$f = 1 - (1-p)^{8000}.$$

Solving for p and substituting $f = 0.1$,

$$p = 1 - (1-f)^{1/8000} = 1 - 0.9^{1/8000}.$$

Numerically, $p \approx 1.3169977732 \cdot 10^{-5}$.

4. Suitable target T . Since $p = T/2^\kappa$, we take $T = p2^\kappa$. Thus

$$T = (1 - 0.9^{1/8000}) 2^{256} \approx 1.5249792368 \cdot 10^{72}.$$

If an integer target is required, we may take

$$T = \lfloor (1 - 0.9^{1/8000}) 2^{256} \rfloor.$$

5. Chernoff interval parameter λ . We want the failure probability to be at most 2^{-128} , so we require

$$4 \cdot 10^{12} e^{-\varepsilon^2 \lambda f / 3} + 4 \cdot 10^{20} 2^{-256} \leq 2^{-128}.$$

Equivalently, it is enough to choose λ such that

$$4 \cdot 10^{12} e^{-\varepsilon^2 \lambda f / 3} \leq 2^{-128} - 4 \cdot 10^{20} 2^{-256}.$$

Taking logarithms gives

$$\lambda \geq \frac{3}{\varepsilon^2 f} \ln \left(\frac{4 \cdot 10^{12}}{2^{-128} - 4 \cdot 10^{20} 2^{-256}} \right).$$

With $\varepsilon = f = 0.1$, this gives $\lambda \geq 353220.463 \dots$. Hence we can take $\lambda = 353221$.

6. Chain growth parameters. We take $s = \lambda = 353221$. The chain velocity is $\tau = (1 - \varepsilon)f = (1 - 0.1)0.1 = 0.09$. Therefore,

$$s = 353221, \quad \tau = 0.09.$$

7. Common prefix parameter. The common prefix parameter is $k = 2\lambda f$. Therefore, $k = 2 \cdot 353221 \cdot 0.1 = 70644.2$. Since k is a number of blocks, we round up and take $k = 70645$.

8. Chain quality parameters. We take $\ell = k = 70645$. For the chain quality parameter we use

$$\mu = 1 - \frac{(1 + \varepsilon)(1 - \delta)(\lambda + 2)}{(1 - \varepsilon)\lambda}.$$

Substituting $\varepsilon = 0.1$, $\delta = 0.75$, and $\lambda = 353221$, we get

$$\mu = 1 - \frac{1.1 \cdot 0.25 \cdot 353223}{0.9 \cdot 353221} \approx 0.6944427143.$$

Exercise 5

Description

When we implemented the Selfish Mining simulation in class using the Monte Carlo method, we implicitly assumed that each block is instantly received by the whole network before the next block is mined. This allowed us to deduce that the probability of the next produced block being adversarial was t/n . However, when we explored the fan-out attack and formalized the bitcoin backbone model, we saw that time discretization into rounds leads to situations in which multiple successful queries can occur in a given round.

Consider a bitcoin backbone execution in the usual synchronous lockstep model. We call a round *adversarially superior* if the round contains an adversarially successful query, but is not a successful round, i.e., there is no honestly successful query. We are interested in the following event E :

An adversarially superior round has occurred strictly prior to the first successful round.

For example, if rounds 1, 2, and 3 contain no successful queries, round 4 contains one adversarially successful query, round 5 contains no successful queries, and round 6 contains one honestly successful query, then the event E has occurred. On the contrary, if rounds 1, 2, and 3 contain no successful queries, round 4 contains an honestly successful query, and round 5 contains only an adversarially successful query, then the event E has not occurred, because the adversarially superior round was after the first successful round.

For simplicity, assume the execution continues for infinite, not just polynomial, duration.

1. Analytically calculate the probability of the event E occurring. Simplify your calculation to closed form, containing no summations.

2. Use Bernoulli's approximation

$$(1 - x)^a \approx 1 - ax$$

for large a and small x to simplify your expression.

3. Find the limit of your probability for $p \rightarrow 0$, i.e., successful queries become rarer and rarer.

Solution

1. Probability of E . Let p be the probability that a single query succeeds. In each round, the honest parties make $q(n - t)$ queries, while the adversarial parties make qt queries.

A round is *successful* if it contains at least one honestly successful query. Hence the probability that a round is successful is

$$h = 1 - (1 - p)^{q(n-t)}.$$

A round is *adversarially superior* if it contains at least one adversarially successful query and no honestly successful query. Therefore its probability is

$$a = (1 - p)^{q(n-t)} (1 - (1 - p)^{qt}).$$

Finally, a round is empty if neither honest nor adversarial queries succeed. This has probability

$$b = (1 - p)^{qn}.$$

The event E occurs if an adversarially superior round appears strictly before the first successful round. Empty rounds do not decide the event; they only delay the process. Thus E occurs if the first non-empty relevant round is adversarially superior.

Hence

$$\Pr[E] = a + ba + b^2a + b^3a + \dots$$

This is a geometric series, so

$$\Pr[E] = \frac{a}{1-b}.$$

Substituting the expressions for a and b , we get

$$\Pr[E] = \frac{(1-p)^{q(n-t)} (1 - (1-p)^{qt})}{1 - (1-p)^{qn}}.$$

2. Simplifying the probability. We now simplify this expression using Bernoulli's approximation

$$(1-x)^a \approx 1 - ax$$

for small x . Applying it to the three terms gives

$$(1-p)^{q(n-t)} \approx 1 - q(n-t)p,$$

$$1 - (1-p)^{qt} \approx qtp,$$

and

$$1 - (1-p)^{qn} \approx qnp.$$

Therefore,

$$\Pr[E] \approx \frac{(1 - q(n-t)p)qtp}{qnp}.$$

Simplifying further we obtain,

$$\Pr[E] \approx \frac{t}{n}(1 - q(n-t)p).$$

3. Considering the limit case when p tends to 0 Finally, taking the limit as $p \rightarrow 0$, we get

$$\lim_{p \rightarrow 0} \Pr[E] = \lim_{p \rightarrow 0} \frac{(1-p)^{q(n-t)} (1 - (1-p)^{qt})}{1 - (1-p)^{qn}}.$$

Using the approximation above, this becomes

$$\lim_{p \rightarrow 0} \frac{(1 - q(n-t)p)qtp}{qnp} = \frac{t}{n}.$$

Thus,

$$\Pr[E] = \frac{(1-p)^{q(n-t)} (1 - (1-p)^{qt})}{1 - (1-p)^{qn}},$$

and in the rare-success limit,

$$\lim_{p \rightarrow 0} \Pr[E] = \frac{t}{n}.$$